

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №4 по курсу
«Операционные системы»

Группа: М8О-211БВ-24

Студент: Захарченко М.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 27.11.25

Москва, 2025

Постановка задачи

Вариант 21.

Задание:

- Требуется создать динамические библиотеки, которые реализуют заданный вариантом функционал. Далее использовать данные библиотеки 2-мя способами:

1. Во время компиляции (на этапе линковки/linking)
2. Во время исполнения программы. Библиотеки загружаются в память с помощью интерфейса ОС для работы с динамическими библиотеками

В конечном итоге, в лабораторной работе необходимо получить следующие части:

- Динамические библиотеки, реализующие контракты, которые заданы вариантом;

- Тестовая программа (программа №1), которая использует одну из библиотек, используя информацию, полученную на этапе компиляции;

- Тестовая программа (программа №2), которая загружает библиотеки, используя только их относительные пути и контракты.

Провести анализ двух типов использования библиотек.

Пользовательский ввод для обеих программ должен быть организован следующим образом:

- Если пользователь вводит команду «0», то программа переключает одну реализацию контрактов на другую (необходимо только для программы №2). Можно реализовать лабораторную работу без данной функции, но максимальная оценка в этом случае будет «хорошо»;

- “1 arg1 arg2 ... argN”, где после “1” идут аргументы для первой функции, предусмотренной контрактами. После ввода команды происходит вызов первой функции, и на экране появляется результат ее выполнения;

- “2 arg1 arg2 ... argM”, где после “2” идут аргументы для второй функции, предусмотренной контрактами. После ввода команды происходит вызов второй функции, и на экране появляется результат ее выполнения.

Подсчёт количества простых чисел на отрезке [a, b] (a, b – натуральные):

Сигнатура функции: `int prime_count(int a, int b);`

- Реализация №1: Наивный алгоритм. Проверить делимость текущего числа на все предыдущие числа.

- Реализация №2: Решето Эратосфена

Отсортировать целочисленный массив:

Сигнатура функции: `int *sort(int *array, size_t n);`

- Реализация №1: Пузырьковая сортировка (^)

- Реализация №2: Сортировка Хоара (за использование `qsort` из `libc` или `std::sort` и его варианты из `libstdc++` будет бан; реализуйте его самостоятельно)

Общий метод и алгоритм решения

Использованные системные вызовы:

- `ssize_t write(int fd, const void *buf, size_t count);` - запись данных.
fd - файловый дескриптор (STDOUT_FILENO = 1 для вывода, STDERR_FILENO = 2 для ошибок); buf - указатель на данные для записи; count - количество байт для записи
- `ssize_t read(int fd, void *buf, size_t count);` - чтение данных.
fd - файловый дескриптор (STDIN_FILENO = 0 для ввода); buf - буфер для сохранения прочитанных данных; count - максимальное количество байт для чтения
- `void *dlopen(const char *filename, int flags);` - загрузка динамической библиотеки.
filename - путь к файлу библиотеки; flags - флаги загрузки:
RTLD_LAZY - отложенное разрешение символов
RTLD_NOW - немедленное разрешение символов
RTLD_LOCAL - символы библиотеки недоступны другим библиотекам
- `void *dlsym(void *handle, const char *symbol);` - получение адреса символа.
handle - указатель на загруженную библиотеку (возвращенный dlopen); symbol - имя символа (функции) для поиска
- `int dlclose(void *handle);` - выгрузка ранее загруженной библиотеки ;handle - указатель на загруженную библиотеку

Системные вызовы для работы с семафорами

- `sem_t *sem_open(const char *name, int oflag, ...)`
Открывает или создает именованный семафор. Возвращает указатель на семафор или SEM_FAILED.
- `int sem_wait(sem_t *sem)`
Ожидает семафор (уменьшает значение на 1). Возвращает 0 при успехе, -1 при ошибке.
- `int sem_post(sem_t *sem)`
Освобождает семафор (увеличивает значение на 1). Возвращает 0 при успехе, -1 при ошибке.
- `int sem_close(sem_t *sem)`
Закрывает семафор. Возвращает 0 при успехе, -1 при ошибке.
- `int sem_unlink(const char *name)`
Удаляет именованный семафор. Возвращает 0 при успехе, -1 при ошибке.

Было создано две библиотеки. Первая реализовывала наивный поиск простых чисел на отрезке [a; b] и пузырьковую сортировку, вторая - решето Эратосфена для поиска простых чисел и быструю сортировку. В первом способе библиотеки подключались во время компиляции (на этапе линковки), во втором способе библиотеки можно подключать в runtime. Поэтому в моей лабораторной работе будет три исполняемых файла демонстрирующих работу с динамическими библиотеками.

Код программы

```
library.h

#ifndef LIBRARY_H
#define LIBRARY_H

#include <stdlib.h>
#include <stdbool.h>
```

```

#include <math.h>
#include <string.h>

typedef int prime_count(int a, int b);
typedef int *sort(int *array, size_t n);

#endif

library_No1.c:
#include "../include/library.h"

#ifdef _MSC_VER
#define EXPORT __declspec(dllexport)
#else
#define EXPORT
#endif

bool isPrime(int number){
    if(number < 2) return false;
    for(int i = 2; i < (int)(sqrt(number) + 1); ++i){
        if(number % i == 0) return false;
    }
    return true;
}

EXPORT int f_prime_count(int a, int b){
    int answer = 0;
    for(int i = a; i <= b; ++i){
        if(isPrime(i) && i >= 2) ++answer;
    }
    return answer;
}

void swap(int *a, int *b){
    int temp = *a;
    *a = *b;
    *b = temp;
}

EXPORT int *f_sort(int *array, size_t n){
    for(int i = 0; i < n - 1; ++i){
        bool change = false;
        for(int j = 0; j < n - i - 1; ++j){
            if(array[j] > array[j+1]){
                change = true;
                swap(&array[j], &array[j+1]);
            }
        }
    }
}

```

```

    }
    if(!change) break;
}
return array;
}

```

library_No2.c:

```
#include "../include/library.h"
```

```
#ifdef _MSC_VER
```

```
#define EXPORT __declspec(dllexport)
```

```
#else
```

```
#define EXPORT
```

```
#endif
```

```
void fillPrimeNumbers(int* primesNumbers, int lastPos){
```

```
    for(int i = 0; i <= lastPos; ++i){
```

```
        primesNumbers[i] = i;
```

```
    }
```

```
    for(int p = 2; p <= lastPos; ++p){
```

```
        if(primesNumbers[p] != 0){
```

```
            for(long long j = (long long)p * p; j <= lastPos; j += p){
```

```
                if(j <= lastPos){
```

```
                    primesNumbers[(int)j] = 0;
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
EXPORT int f_prime_count(int a, int b){
```

```
    int primeNmbers[b+1];
```

```
    fillPrimeNumbers(primeNmbers, b);
```

```
    int answer = 0;
```

```
    for(int i = a; i <= b; ++i){
```

```
        if(primeNmbers[i] != 0 && primeNmbers[i] >= 2) ++answer;
```

```
    }
```

```
    return answer;
```

```
}
```

```
size_t partition(int *arr, size_t left, size_t right) {
```

```
    int mid = arr[(left + right) / 2];
```

```
    int i = left, j = right - 1;
```

```
    int temp;
```

```
    do {
```

```

        while (i < right && arr[i] < mid) {
            ++i;
        }

        while (j >= left && arr[j] > mid) {
            --j;
        }

        if (i <= j) {
            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
            ++i;
            --j;
        }
    } while (i <= j);
    return j;
}

void quicksort(int *arr, size_t left, size_t right) {
    if (right - left < 2) {
        return;
    }

    size_t p = partition(arr, left, right);
    quicksort(arr, left, p + 1);
    quicksort(arr, p + 1, right);
}

EXPORT int *f_sort(int *array, size_t n){
    quicksort(array, 0, n);
    return array;
}

```

programm_No1.c:

```

#include <stddef.h>
#include <stdlib.h>
#include <stdio.h>

#include <unistd.h> // write
#include <dlfcn.h> // dlopen, dlsym, dlclose, RTLD_*

#include "include/library.h"

extern int f_prime_count(int a, int b);
extern int *f_sort(int *array, size_t n);

void print_string(const char *str) {
    write(STDOUT_FILENO, str, strlen(str));
}

```

```

}

int read_line(char *buffer, int max_size) {
    int bytes_read = read(STDIN_FILENO, buffer, max_size - 1);
    if (bytes_read > 0) {
        buffer[bytes_read] = '\0';
        if (buffer[bytes_read - 1] == '\n') {
            buffer[bytes_read - 1] = '\0';
        }
    }
    return bytes_read;
}

void demonstrate_primes() {
    char buffer[128];
    int a, b;

    print_string("Input borders: ");
    read_line(buffer, sizeof(buffer));

    char *token = strtok(buffer, " ");
    if (token) a = atoi(token);

    token = strtok(NULL, " ");
    if (token) b = atoi(token);

    int count = f_prime_count(a, b);

    char num_str[128];
    snprintf(num_str, sizeof(num_str), "Primes Numbers in [a; b]: %d",
count);
    print_string(num_str);
    print_string("\n\n");
}

void demonstrate_sort() {
    char buffer[2048];
    int array[1000];
    int n = 0;

    print_string("Input elements of array: ");
    read_line(buffer, sizeof(buffer));

    char *token = strtok(buffer, " ");
    while (token != NULL) {
        array[n++] = atoi(token);
        token = strtok(NULL, " ");
    }
}

```

```

    }

    int *sorted = f_sort(array, n);

    print_string("Sorted array: ");
    for (int i = 0; i < n; i++) {
        char num_str[32];
        snprintf(num_str, sizeof(num_str), "%d ", sorted[i]);
        print_string(num_str);
    }
    print_string("\n\n");
}

void show_action() {
    print_string("Action\n");
    print_string("1. Count of Primes Numbers in [a; b]\n");
    print_string("2. Sort array\n");
    print_string("3. Exit\n");
    print_string("Choose action: ");
}

int main() {
    char buffer[32];

    while (1) {
        show_action();
        read_line(buffer, sizeof(buffer));

        int command = atoi(buffer);

        switch (command) {
            case 1:
                demonstrate_primes();
                break;
            case 2:
                demonstrate_sort();
                break;
            case 3:
                return 0;

            default:
                print_string("Unknown action. Try again!\n\n");
                break;
        }
    }

    return 0;
}

```



```

}

programm_No2.c:
#include <stddef.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h> // write
#include <dlfcn.h> // dlopen, dlsym, dlclose, RTLD_*

#include "include/library.h"

static prime_count *prime_count_func;
static sort *sort_func;

void print_string(const char *str) {
    write(STDOUT_FILENO, str, strlen(str));
}

int read_line(char *buffer, int max_size) {
    int bytes_read = read(STDIN_FILENO, buffer, max_size - 1);
    if (bytes_read > 0) {
        buffer[bytes_read] = '\0';
        if (buffer[bytes_read - 1] == '\n') {
            buffer[bytes_read - 1] = '\0';
        }
    }
    return bytes_read;
}

void demonstrate_primes() {
    char buffer[128];
    int a, b;

    print_string("Input borders: ");
    read_line(buffer, sizeof(buffer));

    char *token = strtok(buffer, " ");
    if (token) a = atoi(token);

    token = strtok(NULL, " ");
    if (token) b = atoi(token);

    int count = prime_count_func(a, b);

    char num_str[128];

```

```

    snprintf(num_str, sizeof(num_str), "Primes Numbers in [a; b]: %d",
count);
    print_string(num_str);
    print_string("\n\n");
}

void demonstrate_sort() {
    char buffer[2048];
    int array[1000];
    int n = 0;

    print_string("Input elements of array: ");
    read_line(buffer, sizeof(buffer));

    char *token = strtok(buffer, " ");
    while (token != NULL) {
        array[n++] = atoi(token);
        token = strtok(NULL, " ");
    }

    int *sorted = sort_func(array, n);

    print_string("Sorted array: ");
    for (int i = 0; i < n; i++) {
        char num_str[32];
        snprintf(num_str, sizeof(num_str), "%d ", sorted[i]);
        print_string(num_str);
    }
    print_string("\n\n");
}

void show_action() {
    print_string("Action\n");
    print_string("0. Switch library\n");
    print_string("1. Count of Primes Numbers in [a; b]\n");
    print_string("2. Sort array\n");
    print_string("3. Exit\n");
    print_string("Choose action: ");
}

int validate_input(int argc, char* argv[]) {
    if(argc < 3) {
        print_string("Error: Incorrect input\n");
        print_string("Usage: <programm_name> <path_to_lib1>
<path_to_lib2>\n");
        return -1;
    }
}

```

```

    return 0;
}

int main(int argc, char* argv[]) {
    if(validate_input(argc, argv) == -1) return -1;

    char path1[256];
    char path2[256];
    strcpy(path1, argv[1]);
    strcpy(path2, argv[2]);
    int open_lib = 1;
    printf("path1: %s\n", path1);
    printf("path2: %s\n", path2);
    void *library = dlopen(path1, RTLD_LOCAL | RTLD_LAZY);
    if (library == NULL) {
        const char msg[] = "Error: can't open library\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        return -1;
    }

    prime_count_func = dlsym(library, "f_prime_count");
    if(prime_count_func == NULL){
        print_string("Failed to find prime_count implementation");
        return -1;
    }
    sort_func = dlsym(library, "f_sort");
    if(sort_func == NULL){
        print_string("Failed to find sort implementation");
        return -1;
    }

    char buffer[32];

    while (1) {
        show_action();
        read_line(buffer, sizeof(buffer));

        int command = atoi(buffer);

        switch (command) {
            case 0:
                dlclose(library);
                if(open_lib == 1){
                    open_lib = 2;
                    library = dlopen(path2, RTLD_LOCAL | RTLD_LAZY);
                }else if(open_lib == 2){
                    open_lib = 1;

```

```

        library = dlopen(path1, RTLD_LOCAL | RTLD_LAZY);
    }

    if (library == NULL) {
        const char msg[] = "Error: can't open library\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        return -1;
    }

    prime_count_func = dlsym(library, "f_prime_count");
    if(prime_count_func == NULL){
        print_string("Failed to find prime_count
implementation");
        return -1;
    }

    sort_func = dlsym(library, "f_sort");
    if(sort_func == NULL){
        print_string("Failed to find sort implementation");
        return -1;
    }

    print_string("Library switched!\n\n");
    break;
case 1:
    demonstrate_primes();
    break;
case 2:
    demonstrate_sort();
    break;
case 3:
    dlclose(library);
    return 0;

default:
    print_string("Unknown action. Try again!\n\n");
    break;
    }
}

return 0;
}

```

Протокол работы программы

```
matthew@matthews-pc:~/LABS_OPERATING_SYSTEM/LAB_4$ ./compile_demo1
Action
1. Count of Primes Numbers in [a; b]
2. Sort array
3. Exit
Choose action: 1
Input borders: 0 100
Primes Numbers in [a; b]: 25

Action
1. Count of Primes Numbers in [a; b]
2. Sort array
3. Exit
Choose action: 2
Input elements of array: 9 38 213 -21 3211
Sorted array: -21 9 38 213 3211

Action
1. Count of Primes Numbers in [a; b]
2. Sort array
3. Exit
Choose action: 3
```

```
matthew@matthews-pc:~/LABS_OPERATING_SYSTEM/LAB_4$ ./compile_demo2
Action
1. Count of Primes Numbers in [a; b]
2. Sort array
3. Exit
Choose action: 1
Input borders: 0 100
Primes Numbers in [a; b]: 25

Action
1. Count of Primes Numbers in [a; b]
2. Sort array
3. Exit
Choose action: 2
Input elements of array: 122 32 -23 12 0 1 422 2
Sorted array: -23 0 1 2 12 32 122 422

Action
1. Count of Primes Numbers in [a; b]
2. Sort array
3. Exit
Choose action: 3
```

```
matthew@matthews-pc:~/LABS_OPERATING_SYSTEM/LAB_4$ ./runtime_demo ./libprime_sort1.so ./libprime_sort2.so
Action
0. Switch library
1. Count of Primes Numbers in [a; b]
2. Sort array
3. Exit
Choose action: 1
Input borders: 0 100
Primes Numbers in [a; b]: 25

Action
0. Switch library
1. Count of Primes Numbers in [a; b]
2. Sort array
3. Exit
Choose action: 2
Input elements of array: 9 8 7 6 1 2 3 4 5
Sorted array: 1 2 3 4 5 6 7 8 9
```

```
Action
0. Switch library
1. Count of Primes Numbers in [a; b]
2. Sort array
3. Exit
Choose action: 0
Library switched!

Action
0. Switch library
1. Count of Primes Numbers in [a; b]
2. Sort array
3. Exit
Choose action: 1
Input borders: 0 100
Primes Numbers in [a; b]: 25
```

```
Action
0. Switch library
1. Count of Primes Numbers in [a; b]
2. Sort array
3. Exit
Choose action: 2
Input elements of array: 9 8 7 6 5 1 2 3 4 0
Sorted array: 0 1 2 3 4 5 6 7 8 9

Action
0. Switch library
1. Count of Primes Numbers in [a; b]
2. Sort array
3. Exit
Choose action: 3
```

Вывод

В ходе лабораторной работы были созданы две динамические библиотеки реализующие разные алгоритмы поиска простых чисел и сортировок, а именно наивный поиск простых чисел и поиск простых чисел через решето Эратосфена, а также пузырьковая и быстрая сортировки. Также были разработаны интерактивные программы, которые использовали эти библиотеки. Библиотеки были использованы двумя способами:

1. Во время компиляции (на этапе линковки).
2. Во время исполнения программы.

В программе использующей функционал библиотек в runtime была реализована возможность переключения одних контрактов на другие при вызове команды "0". Таким образом, в результате лабораторной работы было изучено и освоено создание динамических библиотек.